


```
solid var(--border); padding: 0.5rem 1.5rem; display: flex; align-items: center;
gap: 0.75rem; flex-wrap: wrap; box-shadow: var(--shadow); } .ctrl-brand { font-
size: 0.7rem; font-weight: 700; letter-spacing: 0.15em; color: var(--accent);
margin-right: auto; text-transform: uppercase; } .ctrl-group { display: flex;
align-items: center; gap: 0.35rem; } .ctrl-label { font-size: 0.68rem; text-
transform: uppercase; letter-spacing: 0.08em; color: var(--text-3); } .ctrl-
select, .ctrl-btn { font-size: 0.78rem; padding: 0.2rem 0.5rem; border: 1px
solid var(--border); border-radius: 0.375rem; background: var(--bg-muted);
color: var(--text-1); cursor: pointer; transition: background 0.15s, border-
color 0.15s; } .ctrl-select:hover, .ctrl-btn:hover { background: var(--accent-
bg); border-color: var(--accent); color: var(--accent); } .ctrl-btn--active {
background: var(--accent) !important; color: #fff !important; border-color:
var(--accent) !important; } /* ?? Cards
????????????????????????????????????????????????????????????????????????? */ .card {
background: var(--bg-card); border: 1px solid var(--border); border-radius:
1rem; padding: var(--d-card); margin-bottom: 2rem; box-shadow: var(--shadow); }
.card-title { font-size: 1.125rem; font-weight: 600; color: var(--text-1);
margin: 0 0 1rem; } .card-subtitle { font-size: 0.95rem; font-weight: 600;
color: var(--text-1); margin: 1.25rem 0 0.625rem; } /* ?? KPI Grid
????????????????????????????????????????????????????????????????????????? */ .kpi-grid {
display: grid; gap: var(--d-gap); grid-template-colum
ns: repeat(auto-fill, minmax(9rem, 1fr)); margin-top: 1.25rem; } .kpi-cell {
background: var(--bg-muted); border-radius: 0.75rem; padding: var(--d-py)
var(--d-px); } .kpi-label { font-size: 0.68rem; text-transform: uppercase;
letter-spacing: 0.1em; color: var(--text-3); margin: 0; } .kpi-value { font-
size: 1.1rem; font-weight: 600; color: var(--text-1); margin: 0.2rem 0 0; } /*
?? Tables ?????????????????????????????????????????????????????????????????? */ .tbl-
wrap { border-radius: 0.75rem; overflow: hidden; border: 1px solid
var(--border); margin-top: 1rem; } table { width: 100%; border-collapse:
collapse; background: var(--bg-card); } thead tr { background: var(--bg-header);
} thead th { padding: var(--d-py) var(--d-px); text-align: left; font-size:
0.7rem; font-weight: 600; text-transform: uppercase; letter-spacing: 0.06em;
color: var(--text-2); } tbody td { padding: var(--d-py) var(--d-px); color:
var(--text-2); border-top: 1px solid var(--border); font-size: 0.875rem; } /*
```

Row-level status tints driven by tr.pass / tr.warn / tr.fail classes. These interact with the CSS token system so they invert correctly in dark mode without any extra JavaScript.

```
tbody tr.pass td { background: var(--pass-bg); color: var(--pass-text); }
tbody tr.warn td { background: var(--warn-bg); color: var(--warn-text); }
tbody tr.fail td { background: var(--fail-bg); color: var(--fail-text); }
```

/* ?? Status Labels

```
/* .status-pass { color: var(--pass-text); font-weight: 700; font-size: 0.7rem; text-transform: uppercase; letter-spacing: 0.08em; }
.status-warn { color: var(--warn-text); font-weight: 700; font-size: 0.7rem; text-transform: uppercase; letter-spacing: 0.08em; }
.status-fail { color: var(--fail-text); font-weight: 700; font-size: 0.7rem; text-transform: uppercase; letter-spacing: 0.08em; }
```

/* ?? Badge

```
/* .badge { display: inline-block; border-radius: 999px; padding: 0.15rem 0.625rem; font-size: 0.68rem; font-weight: 600; letter-spacing: 0.06em; }
.badge-accent { background: var(--accent-bg); color: var(--accent); }
```

/* ??

```
code { background: var(--bg-muted); border-radius: 0.25rem; padding: 0.1em 0.35em; font-size: 0.85em; color: var(--accent); }
```

BLSN Theme ? Light ? Dark ? Auto

Density Compact Comfortable Spacious Audience Executive Manager Engineer BLSN Synthetic Validation Report Theme; light ;Density; comfortable ;Audience; engineer Vehicle P3 Orion Cooling Mode helium_closed_loop Nominal Flow 110 g/s Limits 80?140 g/s Sanity Check Telemetry Status Message PASS [PASS] PASS | Nominal mass flow is bounded by configured limits System Verification: Claimed vs. Actual Claimed Limits: 0 ;|; Actually Tested: 0 ;? STATUS: UNKNOWN Total Runtime 0.00s Pass Ratio 0.0% Fail Ratio 0.0% Skip Ratio 0.0% Tuple Execution Telemetry Targets (seconds)

Metric	Target	Warn	Fail
ci_total_runtime_s	120.0	180.0	240.0
pytest_runtime_s	15.0	25.0	35.0
pipeline_runtime_s	5.0	10.0	15.0
yaml_json_lint_runtime_s	3.0	6.0	9.0

Analytics Dashboard Executive View Engineering View System Priority & Bottleneck Ranking (Bradley-Terry) Pairwise probability model: $P(i > j) = \frac{p_i}{(p_i + p_j)}$ Global Repo Ranking Rank Artifact BT Strength Win Prob vs Top 1

docs/index.html	2.533414	50.00%	2
docs/_includes/metric_card.html	2.317756	47.78%	3
docs/presentation.html	2.309540	47.69%	4

docs/_includes/plotly_render.html 2.246689 47.00% 5 docs/_layouts/default.html
2.161844 46.04% 6 docs/blsn_report.pdf 0.795024 23.89% 7
docs/visuals/plotly_payloads.json 0.481794 15.98% 8
docs/experimental_design_matrix.json 0.417307 14.14% 9
docs/historical_telemetry_ledger.json 0.409257 13.91% 10
docs/federation/federation_cherry_pick.json 0.382788 13.13% Inter-Category
Ranking Rank Category Win Prob vs Top 1 html 50.00% 2 json 15.19% 3 md 10.52% 4
pdf 6.43% 5 yaml 3
.18% 6 yml 2.87% QPLANT Requirements Ranking Bridge Rank Requirement Win Prob vs
Top 1 ci_total_runtime_s 50.00% 2 pytest_runtime_s 50.00% 3 pipeline_runtime_s
50.00% 4 yaml_json_lint_runtime_s 50.00% Wave 12: SPC & Telemetry Stats
Runtime Drift 0.0000s Prev Moving Avg 0.0000s Total Runs 2 Success Rate 100.0%
DoE Factors ? A: python_version · B: os_type · C: load_parameter Principal
Component Analysis (PCA Proof) Principal Component Eigenvalue Explained Variance
Highest Weight Variable PC1 0.000000 0.00% total_tests PC2 0.000000 0.00%
skip_rate Metric Markdown Slugs Ci Total Runtime S Pytest Runtime S Pipeline
Runtime S Yaml Json Lint Runtime S Ci Total Runtime S Metric ID:
`ci\_total\_runtime\_s` Target Threshold: **120.0s** Warn Threshold: **180.0s**
Fail Threshold: **240.0s** Pytest Runtime S Metric ID: `pytest\_runtime\_s` Target
Threshold: **15.0s** Warn Threshold: **25.0s** Fail Threshold: **35.0s**
Pipeline Runtime S Metric ID: `pipeline\_runtime\_s` Target Threshold: **5.0s**
Warn Threshold: **10.0s** Fail Threshold: **15.0s** Yaml Json Lint Runtime S
Metric ID: `yaml\_json\_lint\_runtime\_s` Target Threshold: **3.0s** Warn Threshold:
6.0s Fail Threshold: **9.0s** Mermaid System Flow Diagram graph LR;
SSOT[SSOT Config] --> PIPELINE[Pipeline Execution]; PIPELINE -->
MARKDOWN[Markdown Slug Pages]; PIPELINE --> INDEX[Index HTML]; PIPELINE -->
SLIDES[Slides HTML]; INDEX --> PDF[PDF Export]; /* ?? BLSN Report UI Controls
?? All user preferences are persisted to
localStorage under the "blsn_" namespace so they survive hard reloads without
server round-trips. Theme: light | dark | auto "auto" resolves to light or dark
via prefers-color-scheme and keeps the attribute updated when the OS preference
changes (e.g. switching between light and dark system themes). Density: compact
| comfortable | spacious Controls the --d-py / --d-px / --d-gap / --d-card CSS
tokens that scale table cell padding and card gutters without reflowing the pa

```
ge. Audience: executive | manager | engineer CSS [data-audience] selectors hide
.aud-manager and .aud-engineer sections for less technical readers. Changing
this value simply updates the data attribute; no DOM nodes are added or removed.
Plotly charts are re-rendered via Plotly.react() rather than Plotly.newPlot()
when the theme changes so the existing DOM element is reused and transitions
feel instant. ?? */ (function () { 'use strict'; const PREF_NS = 'blsn_'; const
getPref = (k, def) => localStorage.getItem(PREF_NS + k) || def; const setPref =
(k, v) => localStorage.setItem(PREF_NS + k, v); const root =
document.documentElement; /* Resolve the "auto" theme sentinel using the OS
preference. */ function resolveTheme(raw) { if (raw !== 'auto') return raw;
return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' :
'light'; } /* Apply a data-attribute and keep its matching in sync. */ function
applyAttr(attr, selectId, value) { root.setAttribute('data-' + attr, value);
const el = document.getElementById(selectId); if (el) el.value = value;
setPref(attr, value); } /* ?? Plotly theme-aware rendering
????????????????????????????????????????? */ let _payload = null; /* cached payload to
allow re-render on theme change */ /* Patch a Plotly chart layout for dynamic
font/grid colours. Backgrounds stay transparent (set by Python); only text and
grid colours are adjusted so light/dark switching works seamlessly. */ function
patchLayout(chart, themeRaw) { const isDark = resolveTheme(themeRaw) === 'dark';
const gridColor = isDark ? '#334155' : '#e2e8f0'; const fontColor = isDark ?
'#94a3b8' : '#475569'; return Object.assign({}, chart.layout || {}, {
paper_bgcolor: 'rgba(0,0,0,0)', plot_bgcolor: 'rgba(0,0,0,0)', font: { color:
fontColor }, xaxis: Object.assign({}, (chart.layout || {}).xaxis, { gridcolor:
gridColor, zerolinecolor: gridColor }), yaxis: Object.assign({}, (chart.layout
|| {}).yaxis, { gridcolor: gridColor, zerolinecolor: gridColor }), }); } /* Re-
render both
Plotly charts with the resolved theme. Reads from audience-structured payload
(executive / engineering). */ function rerenderPlotly(themeRaw) { if
(!window.Plotly || !_payload) return; const audience = _payload.audience || {};
const tel = (audience.executive || {}).telemetry_combo || {}; const pca =
(audience.engineering || {}).pca_bottlenecks || {}; Plotly.react('plotly-
telemetry-chart', tel.data || [], patchLayout(tel, themeRaw));
Plotly.react('plotly-pca-chart', pca.data || [], patchLayout(pca, themeRaw)); }
```

```

/* ?? Chart-view audience tab toggle ????????????????????????????????? */
window.switchChartView = function (view) { const viewExec =
document.getElementById('view-executive'); const viewEng =
document.getElementById('view-engineering'); const btnExec =
document.getElementById('btn-exec'); const btnEng =
document.getElementById('btn-eng'); if (!viewExec || !viewEng) return; const
isExec = view === 'executive'; viewExec.style.display = isExec ? 'block' :
'none'; viewEng.style.display = isExec ? 'none' : 'block'; if (btnExec)
btnExec.classList.toggle('ctrl-btn--active', isExec); if (btnEng)
btnEng.classList.toggle('ctrl-btn--active', !isExec); /* Trigger resize so
Plotly fills a div that was previously hidden. */ window.dispatchEvent(new
Event('resize')); }; /* ?? Public setters wired to onchange handlers
???????????????????? */ window.setTheme = function (v) { /* Apply the resolved
value to data-theme but store the raw choice (including "auto") so the select
always reflects user intent. */ root.setAttribute('data-theme',
resolveTheme(v)); const el = document.getElementById('ctrl-theme'); if (el)
el.value = v; setPref('theme', v); rerenderPlotly(v); }; window.setDensity = (v)
=> applyAttr('density', 'ctrl-density', v); window.setAudience = (v) =>
applyAttr('audience', 'ctrl-audience', v); /* ?? Initialise from saved
preferences (fall back to SSOT defaults) */ const initTheme = getPref('theme',
'light'); const initDensity = getPref('density', 'comfortable'); const
initAudience = getPref('audience', 'engineer'); /* Apply theme without going
through the setter so the select shows "auto" rather than the resolved value
when stored as "auto". */ root.setAttribute('data-theme',
resolveTheme(initTheme)); const themeEl = document.getElementById('ctrl-theme');
if (themeEl) themeEl.value = initTheme; applyAttr('density', 'ctrl-density',
initDensity); applyAttr('audience', 'ctrl-audience', initAudience); /* ?? Plotly
payload fetch ????????????????????????????????????????????? */ const emptyLayout = ()
=> ({ paper_bgcolor: 'rgba(0,0,0,0)', plot_bgcolor: 'rgba(0,0,0,0)', xaxis: {
visible: false }, yaxis: { visible: false }, annotations: [{ text: 'No Plotly
payload available', showarrow: false, xref: 'paper', yref: 'paper', x: 0.5, y:
0.5, }], }); fetch('visuals/plotly_payloads.json') .then((r) => (r.ok ? r.json()
: Promise.reject(new Error('HTTP ' + r.status)))) .then((payload) => { _payload
= payload; rerenderPlotly(getPref('theme', 'light')); }) .catch(() => { const el

```

```
= emptyLayout(); Plotly.newPlot('plotly-telemetry-chart', [], el);
Plotly.newPlot('plotly-pca-chart', [], el); }); /* Keep "auto" theme in sync
when the OS preference changes at runtime (e.g. user switches system dark mode
while the tab is open). */ window.matchMedia('(prefers-color-scheme:
dark)').addEventListener('change', () => { const stored = getPref('theme',
'light'); if (stored === 'auto') { root.setAttribute('data-theme',
resolveTheme('auto')); rerenderPlotly('auto'); } }); });
```